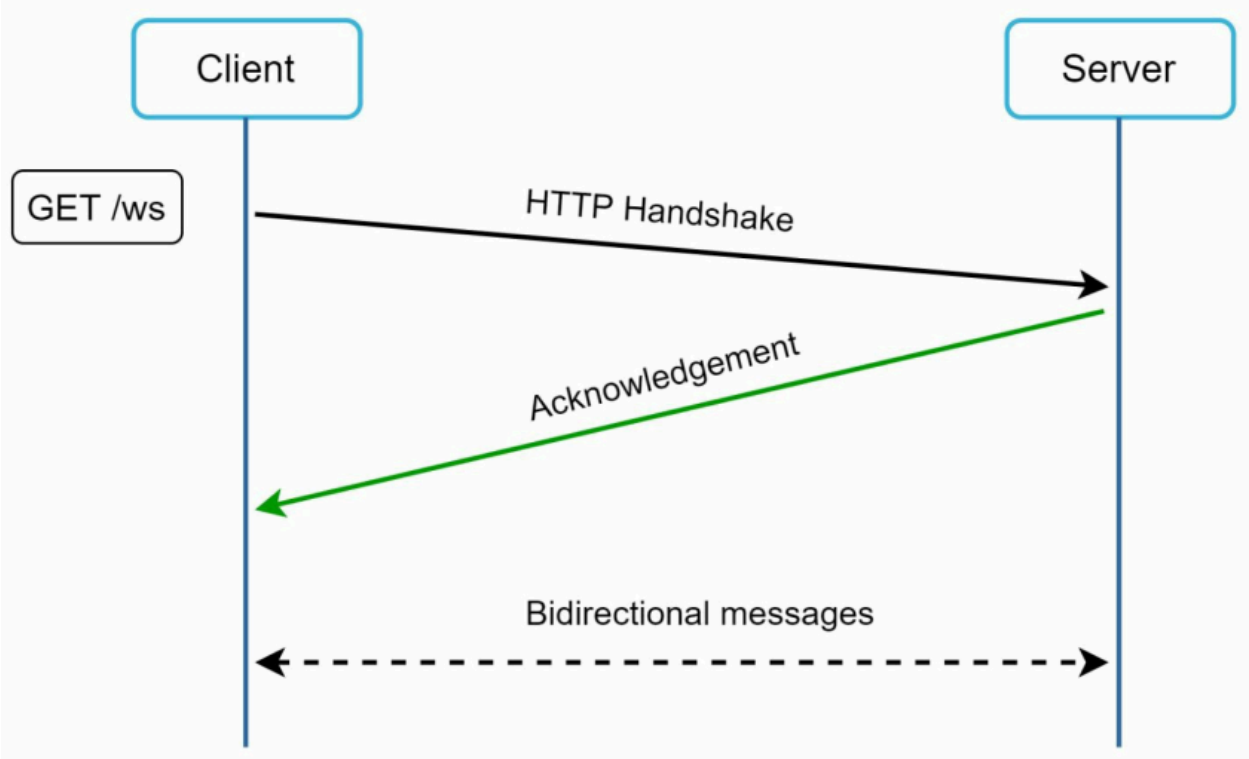
WebSocket in Chat System

1. What is WebSocket?

WebSocket is the most widely used solution for:

```
Real-time bidirectional communication
```

between client and server.



2. Why WebSocket is Needed

Earlier approaches like:

- Polling
- Long Polling

had several problems:

- High latency
- Frequent reconnections
- Large server overhead
- Inefficient communication

WebSocket solves these issues by maintaining a:

```
None
Persistent Full-Duplex Connection
```

3. How WebSocket Works

Step 1: Connection Starts as HTTP

A WebSocket connection initially begins as:

```
None
HTTP Connection
```

Step 2: Upgrade Handshake

Using a special handshake:

```
None
HTTP → WebSocket Upgrade
```

the protocol switches from HTTP to WebSocket.

Step 3: Persistent Connection Established

Once upgraded:

- Connection remains open
- No repeated reconnections required
- Both client and server can send data anytime

4. Key Characteristics of WebSocket

Bidirectional Communication

Both sides can communicate independently.

```
None
Client ↔ Server
```

Unlike HTTP:

- Server does not wait for client requests

Persistent Connection

Single connection remains active for long duration.

Benefits:

- Reduced latency
- Faster message delivery
- Fewer TCP handshakes

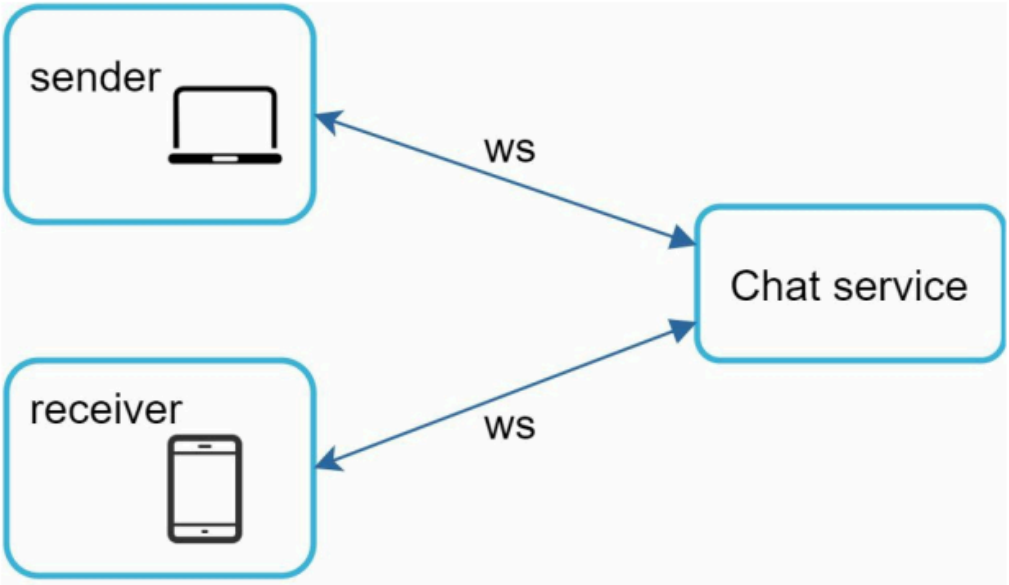
Real-Time Updates

Server can instantly push messages to users.

Ideal for:

- Chat applications
- Online gaming
- Live collaboration
- Notifications

5. WebSocket in Chat Systems



In modern chat systems:

```
None
WebSocket is used for BOTH sending and receiving messages
```

6. Earlier Design vs WebSocket Design

Earlier Approach

```
None

Sender → HTTP

Receiver → Long Polling
```

Two different protocols increased complexity.

Improved Approach

```
None

Sender ↔ WebSocket ↔ Receiver
```

Single protocol for both directions.

7. Advantages of Using WebSocket Everywhere

Simpler Architecture

Using one protocol simplifies:

- Backend implementation
- Client implementation
- Message routing

Low Latency

Messages are delivered almost instantly.

Reduced Overhead

No repeated polling requests.

Better User Experience

Supports:

- Typing indicators
- Instant delivery
- Online presence
- Real-time updates

8. Firewall Compatibility

WebSocket connections usually work even behind firewalls.

Why?

Because WebSocket commonly uses:

```
None
Port 80 (HTTP)

Port 443 (HTTPS)
```

These ports are already allowed in most networks.

9. Important Server-Side Challenge

Since WebSocket connections are:

```
None
Persistent
```

servers must manage many long-lived connections efficiently.

10. Connection Management Challenges

At massive scale:

```
None
Millions of active WebSocket connections
```

can exist simultaneously.

Server must handle:

- Connection lifecycle
- Heartbeats
- Disconnect detection
- Reconnection
- Load balancing

efficiently.

11. Benefits of WebSocket for Chat Apps

Feature	Benefit

Persistent connection	Faster communication
Bidirectional messaging	Real-time updates
Low latency	Instant message delivery
Reduced overhead	Better scalability
Single protocol	Simpler implementation

12. High-Level Communication Flow

```
None
Client A ↔ Chat Server ↔ Client B
```

Using WebSocket:

1. Sender sends message
2. Chat server receives message
3. Server instantly pushes message to receiver
4. Receiver gets message in real time

Key Takeaway

WebSocket is the preferred protocol for modern chat systems because it provides:

- Persistent communication
- Real-time bidirectional messaging

- Low latency
- Efficient resource usage

Using WebSocket for both sender and receiver sides:

- Simplifies architecture
- Improves scalability
- Enhances user experience

Efficient connection management becomes critical at large scale because millions of persistent connections must be maintained simultaneously.